Self-Supervised LLM Fine-Tuning Using DSP-RL

Samit Ahlawat

Abstract

Fine-tuning LLMs to specific domains has been demonstrated to improve the quality of LLM responses for a range of tasks. This has spurred the development of fine-tuning methods including full-model fine-tuning, data-efficient fine-tuning and parameter-efficient fine-tuning among the supervised fine-tuning methods. Another branch of effort has targeted reinforcement-learning to align LLM outputs closer to human preferences or to steer them away from producing biased content. This work presents a new self-supervised fine-tuning method that integrates the DSP-RL approach with DoRA parameter-efficient fine-tuning method to facilitate sharpening of reasoning skills in LLMs. NLGraph dataset is used to identify a cohort of hard graph problems, without using the final answer available in the dataset. Two copies of an LLM are created – a proponent and an opponent. Proponent answers a question while opponent attempts to identify reasoning related shortcomings in the proponent's response. Proponent uses this critique as an additional cue to answer the original prompt a second time. RoBerta model is used to gauge the level of similarity between the two proponent responses - one produced by itself and the other produced using opponent's critique. Responses that have similarity score below a threshold are selected as examples for fine-tuning, with the second proponent response serving as the model response during DoRA supervised fine-tuning. Reasoning improvement from the fine-tuned model is demonstrated.

Keywords: Self-supervised fine-tuning, DoRA, DSP-RL, reasoning skills in LLM

1. Introduction

Large-language models (LLMs) often struggle with complex graph-related reasoning tasks due to the challenge of integrating graph-specific concepts such as nodes, edges, connections and weights with general reasoning skills

such as inductive reasoning (Hu *et. al.*, 2025). Several methods have been proposed to mitigate this shortcoming, such as graph neural networks (Scarselli *et. al.*, 2009) and using a multi-agent collaborative system (Hu *et. al.*, 2025). Graph neural networks (GNNs) have been proposed to model graphs using deep neural networks. However, due to their graph-specific structure, they are not as versatile in picking general reasoning skills from large text corpora on which LLMs are trained. Consequently, they lack the general reasoning skills of LLMs. In addition, they need special adaptation for performing convolutions (Kipf and Welling, 2017).

Supervised fine-tuning has been used extensively to augment pre-trained LLMs with domain-specific information, leading to improved zero-shot performance (Wei *et. al.*, 2022). This has led to the development of a range of supervised fine-tuning methods from full parameter fine-tuning to data-efficient and parameter-efficient fine-tuning. Supervised fine-tuning (SFT) methods rely on the availability of labeled datasets, which can prove to be a bottleneck in new applications. Reinforcement-learning based methods such as RLHF leverage a model that evaluates the degree of conformity of LLM response with some desirable attribute such as alignment with human preferences to formulate a reward function. This function is used to fine-tune an LLM by maximizing expected discounted sum of rewards using reinforcement learning.

In order to improve reasoning skills of LLMs, several approaches have been investigated by researchers. Patil and Jordan (2025) provide a helpful taxonomy of such efforts by categorizing them into the following classes.

1. Prompt-based methods: These methods leverage the existing reasoning skills of pre-trained LLMs by providing explicit instructions in the prompt to perform actions that aid in the development of coherent reasoning. Prominent methods in this category include:
   (a) Chain-of-thought (CoT) prompting: The LLM is asked to produce a sequence of reasoning steps to answer the prompt (Wei *et. al.*, 2023). This aids in the development of a logical and verifiable approach for solving the problem, enabling the LLM to learn reasoning skills.
   (b) Self-consistency prompting: In this method, the LLM is prompted to produce multiple chain-of-thoughts and to select the most common reasoning path using a majority-vote (Wang *et. al.*, 2023).
   (c) Tree-of-thought prompting: This method involves prompting the LLM to explore a complex reasoning task using a tree of reasoning

steps, allowing the model to backtrack and formulate complex reasoning paths to solve the problem (Yao *et. al.*, 2023).

(d) Program-of-thought prompting: This method separates the numerical workload of the solution from the reasoning workload. The LLM handles the reasoning task, producing executable code that handles the numerical computation component of th problem (Chen *et. al.*, 2023).

(e) Instruction tuning: This method, proposed by Wei *et. al.* (2022), involves fine-tuning an LLM on different categories of tasks using natural language based instructions. This improves zero-shot performance of the fine-tuned LLM on new tasks.

2. LLM architecture based methods: This method involves augmenting LLM training using specialized neural networks, such as Graph Neural Networks (GNN) for graph based tasks.
3. Training LLMs with specialized reasoning datasets: This approach recommends using specialized datasets for training LLMs to aid in sharpening their reasoning skills.
4. Retrieval-augmented reasoning: This method is based on retrieval augmented generation (RAG) and involves augmenting the prompt with task-specific knowledge, often found by searching a vector database.
5. Self-supervised and reinforcement learning: This method has been widely used to impart certain desirable characteristics to LLM output, such as alignment with human preferences (RLHF), or to steer LLM away from producing biased or harmful content. Reinforcement learning methods such as REINFORCE, PPO, GRPO, GSPO and DPO have been used to fine-tune LLMs (Lambert, N., 2026).
6. Memory-augmented neural networks (MANN): This methodology leverages external memory modules to record facts and assist an LLM to retrieve relevant knowledge at the time of generating response to a prompt. Khosla *et. al.* (2023) provide a brief survey on the topic.

This work introduces a new method to combine debate-and-self-play reinforcement learning (DSP-RL) to perform a parameter-efficient fine-tuning of LLM using DoRA for improving the reasoning skills of an LLM. Debate-and-self-play reinforcement learning uses a proponent-opponent duo, with proponent answering a prompt and opponent attempting to verify the solution and find reasoning flaws in proponent's response (Srivastava and Aggarwal, 2025). The proponent then uses the opponents critique to modify its response.

DoRA (Weight-decomposed low-rank adaptation) is a parameter-efficient supervised fine-tuning method introduced by Mao *et. al.* (2024) that improves upon the performance of LoRA (low-rank adaption, Hu *et. al.*, 2021) by decomposing the weight correction into direction and magnitude. This results in fine-tuned models that are closer in prediction capabilities to those obtained using full parameter fine-tuning.

By combining DSP-RL method with DoRA fine-tuning method, we propose a self-supervised parameter-efficient fine-tuning method that does not depend on the availability of curated datasets with ground-truth responses. It uses a proponent-opponent pair to improve upon a response to a reasoning question. It quantifies the degree of difference between the responses using Roberta similarity score and selects the responses with difference above a specified threshold. The improved responses are then used to perform a supervised parameter efficient fine-tuning using DoRA. Improvement in reasoning skills after fine-tuning is demonstrated.

This paper is structured as follows: Section 2 describes the dataset used and the methodology of using DSP-RL for performing DoRA based supervised fine-tuning for an LLM. Section 3 details the LLM models used and run-settings used in testing the proposed fine-tuning methodology. Section 4 presents the results and section 5 concludes with a summary of key findings and directions for future research.

2. Data and Methodology

We use NLGraph dataset (Wang *et. al.*, 2024) to identify a set of hard graph problems. The reason for selecting graph problems is that they can be computationally hard, are easy to create progressively harder versions with the same underlying logic – for instance, by increasing the number of vertices and edges – and have a verifiable answer. LLMs have a difficult time solving hard graph problems. This has led to the development of Graph Neural Networks (GNN) that are uniquely adapted to represent the graph topology and solve graph problems. However, they lack the general reasoning and natural language skills of large language models.

We use arnir0/Tiny-LLM (Hugging Face, 2024) as the LLM model for fine-tuning. This is one of the smallest LLMs available that could be loaded onto available GPU processors in training mode, with around 13 million free parameters. Even though the majority of LLM weights are frozen, with only a small fraction of weights being modifiable during parameter-efficient

fine-tuning under DoRA, we could not load larger models such as Gemma-4 E2B (Gemma Team, 2026). However, arnir0/Tiny-LLM was not able to generate coherent responses for any of the graph problems. As a result, we use Gemma 4 E2B as the proponent and opponent model to generate an improved response to hard graph problems. Gemma 4 E2B has 2.3 billion effective parameters. Since this model is not used in training, we run it using ollama.

We select a set of hard graph problems from NLGraph dataset. The proponent model is prompted to solve the problem first. Opponent model is then provided the problem along with proponent model's response and is asked to find reasoning flaws and potential problems in the proponent's response. Opponent's critique is then fed back to the proponent model, which gets a second shot at generating an improved response. This cycle can be repeated N times. In this work, we use $N = 1$. Due to the fact that we want to create an unsupervised learning methodology, we do not use final response for the problem from NLGraph dataset. Instead, we evaluate the degree of similarity between the original proponent's response and the final proponent's response using RoBERTa model for text sequence classification (Liu *et. al.*, 2019). RoBERTa model accepts a reference response and a model-generated response and produces a probability score quantifying the similarity between the two responses. If the similarity score is below a specific threshold – set to 0.6 in this work – the prompt and the final proponent response are added to the dataset for fine-tuning the model. This dataset represents the problems where opponent's critique, when added to the original prompt, caused the proponent LLM to revise its response. Target LLM is then fine-tuned using DoRA method using the dataset created.

The full training algorithm is described in 2.

3. Model Details and Testing

airnir0/Tiny-LLM has 13,000,032 free parameters. After applying DoRA fine-tuning for query and value projection modules (by setting target_modules to q_proj and v_proj), the number of trainable parameters are 11,040, which is around 0.08% of the original number of parameters.

4. Results

The results show limited improvement, with the LLM learning to regurgitate the problem. However this is due to the limited size of original LLM.

Algorithm 1 Fine-Tune LLM Using DSP-RL and DoRA

Require: Proponent LLM P, opponent LLM O, target LLM T, number of turns N, RoBERTa model for similarity score R and similarity score threshold s.

for Each hard problem in dataset **do**
    Prompt the proponent LLM to solve the problem.
    for each i from 0 to N **do**
        Provide the proponent's response to opponent LLM and prompt it to spot reasoning flaws and verify the response.
        Provide the opponent's critique to the proponent and prompt it to solve the problem again.
    end for
    Evaluate the degree of similarity between original and final proponent response. Use RoBERTa model.
    If similarity is below s, add the prompt and the final proponent response to dataset $\mathcal{D}$.
end for
Fine-tune target LLM T using the dataset $\mathcal{D}$ with DoRA methodology.
Merge the fine-tuned weights to obtain the final LLM.
Prompt the fine-tuned LLM to solve hard graph problems.
Compute the similarity score between the new LLM response and final proponent response for the same graph problem. This is the incremental gain due to DSP-RL and DoRA fine-tuning.

With the availability of more compute resources, a realistic size LLM such as Gemma 4 E2B could have been tested.

5. Conclusion

6. Code

Code for reproducing the results presented in this work can be found on the author's GitHub repository `https://github.com/samit-ahlawat/MoE_Encoder_Return_Pred`.

References

[1] Hu, U., Lei, R., Huang, X., Wei, Z., Liu, Y., 2025. Scalable and Accurate Graph Reasoning with LLM-Based Multi-Agents, *Association for the Advancement of Artificial Intelligence 2026 Workshop WMAC*, *arXiv:2410.05130 [cs.AI]*, `https://arxiv.org/abs/2410.05130v3`.

[2] Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., Monfardini, G., 2009. *The graph neural network model. IEEE Transactions on Neural Networks*, 20(1), 61-80.

[3] Kipf, T. N., Welling, M., 2017. Semi-Supervised Classification with Graph Convolutional Networks, *International Conference on Learning Representations (ICLR) 2017*.

[4] Wei, J., Bosma, M., Zhao, V., Y., Guu, K., Yu, A., W., Lester, B., Du, N., Dai, A., M., Le, Q., V., 2022. Finetuned Language Models Are Zero-Shot Learners, *arXiv:2109.01652 [cs.CL]*, `https://arxiv.org/abs/2109.01652`.

[5] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D., 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, *arXiv:2201.11903 [cs.CL]*, `https://arxiv.org/abs/2201.11903`.

[6] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D., 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models, *arXiv:2203.11171 [cs.CL]*, `https://arxiv.org/abs/2203.11171`.

[7] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., Narasimhan, K., 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models, *arXiv:2305.10601 [cs.CL]*, {https://arxiv.org/abs/2305.10601}.\bibitem{POT2023}Chen,W.,Ma,X.,Wang,X.,Cohen,W.W.,2023.ProgramofThoughtsPrompting: DisentanglingComputationfromReasoningforNumericalReasoningTasks, \emph{TransactionsonMachineLearningResearch,2023}, \urlhttps://arxiv.org/abs/2211.12588.

[8] Lambert, N., 2026. Reinforcement Learning from Human Feedback, *arXiv:2504.12501 [cs.LG]*, {https://arxiv.org/abs/2504.12501}.\bibitem{MANN2023}Khosla,S.,Zhu,Z.,He,Y.,2023.SurveyonMemory-AugmentedNeuralNetworks: CognitiveInsightstoAIApplications,\emph{arXiv:2312.06141[cs.AI]},\urlhttps://arxiv.org/abs/2312.06141.

[9] Srivastava, S., S., Aggarwal, V., 2025. A Technical Survey of Reinforcement Learning Techniques for Large Language Models, *arXiv:2507.04136v1 [cs.AI]*, {https://arxiv.org/html/2507.04136v1}.\bibitem{DoRA2024}Mao,Y.,Huang,K.,Guan,C.,Bao,G.,Mo,F.,Xu,J.,2024.DoRA: EnhancingParameter-EfficientFine-TuningwithDynamicRankDistribution, \emph{arXiv:2405.17357[cs.CL]},\urlhttps://arxiv.org/abs/2405.17357.

[10] Hu, J. E., Shen, Y., L., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., A., Wang, L., Chen, W., Z, 2021. LoRA: Low-Rank Adaptation of Large Language Models, *arXiv:2106.09685*, https://arxiv.org/abs/2106.09685.

[11] Wang, H., Feng, S., He, T., Tan, Z., Han, X., Tsvetkov, Y., 2024. Can Language Models Solve Graph Problems in Natural Language?, *arXiv:2305.10037 [cs.CL]*, https://arxiv.org/abs/2305.10037.

[12] Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., Amodei, D., 2017. Deep Reinforcement Learning from Human Preferences *Advances in Neural Information Processing Systems (NeurIPS)*, https://arxiv.org/abs/1706.03741, 30, 4299-4307.

[13] Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., Christiano, P., Kreuger, J., 2019. Fine-Tuning Language Models

from Human Preferences, *arXiv:1909.08593 [cs.CL]*, https://arxiv.org/abs/1909.08593.

[14] arnir0, 2024. Tiny-LLM, *Hugging Face*, https://huggingface.co/arnir0/Tiny-LLM.

[15] Gemma Team, 2026. Gemma 4 Technical Report, *arXiv:2607.02770*.

[16] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv:1907.11692*.